



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

**Centro de
e-Learning**

UNIDAD DIDÁCTICA II

PYTHON PASO A PASO

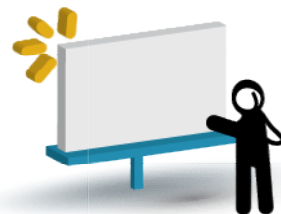
Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning

Modulo I – Introducción

Unidad III – Adquisición de datos y visualización.



Presentación:

Una tarea muy habitual es realizar la toma de datos y luego visualizarlos, en python contamos con dos herramientas muy útiles para realizar esta tarea:

Pandas: Permite tomar los datos provenientes de diferentes formatos, en sí es una biblioteca de código abierto con licencia BSD que proporciona estructuras de datos de alto rendimiento y fáciles de usar. Es un proyecto patrocinado por NumFOCUS.

Matplotlib es una herramienta imprescindible en el análisis ya que nos permite presentar la información en una variedad enorme de formatos tanto de 2D como 3D.

En el transcurso de esta unidad veremos cómo sacar provecho de ambas librerías.



Objetivos:

Que los participantes:

Puedan obtener los datos de diferentes fuentes de procedencias.

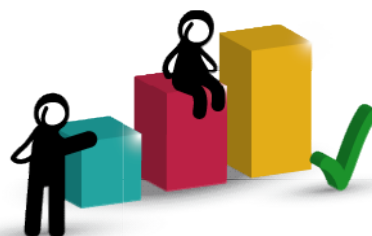
Aprendan a visualizar los datos adquiridos con pandas.

Exploreen diferentes formatos de representación de datos.



Bloques temáticos:

- 1.- Instalación.
- 2.- Tomar datos con pandas.
3. Visualización con Matplotlib
- 4.- Mapa de colores.



Consignas para el aprendizaje colaborativo

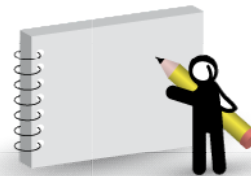
En esta Unidad los participantes se encontrarán con diferentes tipos de actividades que, en el marco de los fundamentos del MEC*, los referenciarán a tres comunidades de aprendizaje, que pondremos en funcionamiento en esta instancia de formación, a los efectos de aprovecharlas pedagógicamente:

- Los foros proactivos asociados a cada una de las unidades.
- La Web 2.0.
- Los contextos de desempeño de los participantes.

Es importante que todos los participantes realicen algunas de las actividades sugeridas y compartan en los foros los resultados obtenidos.

Además, también se propondrán reflexiones, notas especiales y vinculaciones a bibliografía y sitios web.

El carácter constructivista y colaborativo del MEC nos exige que todas las actividades realizadas por los participantes sean compartidas en los foros.



Tomen nota

Las actividades son opcionales y pueden realizarse en forma individual, pero siempre es deseable que se las realice en equipo, con la finalidad de estimular y favorecer el trabajo colaborativo y el aprendizaje entre pares. Tenga en cuenta que, si bien las actividades son opcionales, su realización es de vital importancia para el logro de los objetivos de aprendizaje de esta instancia de formación. Si su tiempo no le permite realizar todas las actividades, por lo menos realice alguna, es fundamental que lo haga. Si cada uno de los participantes realiza alguna, el foro, que es una instancia clave en este tipo de cursos, tendrá una actividad muy enriquecedora.

Asimismo, también tengan en cuenta cuando trabajen en la Web, que en ella hay de todo, cosas excelentes, muy buenas, buenas, regulares, malas y muy malas. Por eso, es necesario aplicar filtros críticos para que las investigaciones y búsquedas se encaminen a la excelencia. Si tienen dudas con alguno de los datos recolectados, no dejen de consultar al profesor-tutor. También aprovechen en el foro proactivo las opiniones de sus compañeros de curso y colegas.



1. Instalación

Antes de comenzar a utilizar pandas y matplotlib, debemos instalarlas en nuestra distribución de python, ya que son librerías de terceros que no vienen por defecto.

La instalación es simple, solo abrimos una terminal y ejecutamos los siguientes comandos.

```
pip install pandas  
pip install matplotlib
```

1.1. Pandas

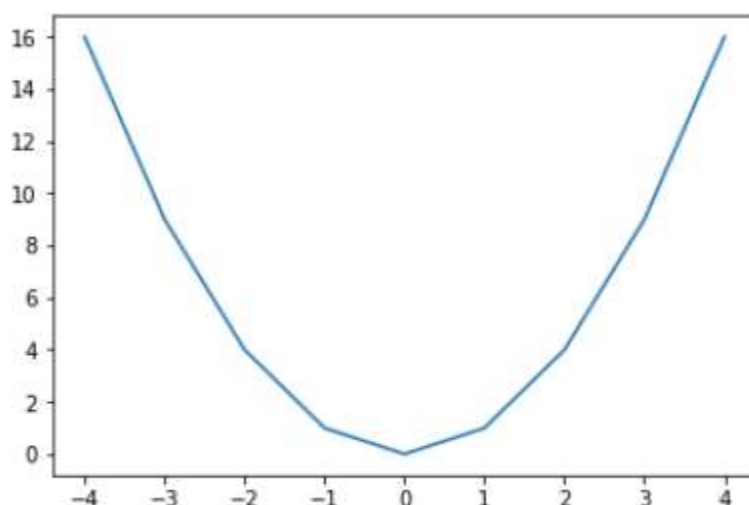
Pandas es una librería de python que nos permite obtener datos de distinta procedencia como html, csv, json, pickle, sql. Un ejemplo simple de cómo cargar un archivo csv podría ser:

```
import pandas as pd  
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1")  
dat_csv
```

1.2. Matplotlib

Matplotlib es un paquete de Python para la visualización de datos, que nos permite trabajar con diferentes formatos de representación. Matplotlib nos brinda una interfaz simple orientada a objetos en la cual podemos trazar desde gráficos simples a una amplia variedad de gráficos e incluso aplicarlo en el desarrollo web. Podemos ver a continuación un ejemplo en donde a partir de las listas de datos x e y, creamos el gráfico con la ayuda de plot() y show.

```
import matplotlib.pyplot as plt  
x = list(range(-4 , 5))  
y = [i**2 for i in x]  
plt.plot(x,y)  
plt.show()
```



2. Fuentes de datos

1. Tomar datos con pandas

Importar datos con pandas es muy simple, a continuación analizaremos diferentes fuentes de procedencia:

Formato CSV

Para tomar datos de un archivo csv, podemos partir de una hoja de Excel que posea una tabla como la siguiente:

id	Nombre	Sexo
1	Juan	m
2	Pedro	m
3	Anna	f
4	Julieta	f
5	Pablo	m
6	Ramón	m
7	Gaby	f



Es necesario que dejemos solo una hoja en el libro y lo guardemos con formato '**csv delimitado por comas**'

Nota: Cuando tenemos configurada la maquina en español la separación entre datos es tomada por punto y coma (;) en lugar de coma (,). En este caso podemos utilizar algún editor de texto como notepad++ y sustituir todos los punto y comas (;) por comas (,).

Si ahora abrimos una hoja de jupyter podemos importar el archivo mediante el método `read_csv` de `pandas`.

```
import pandas as pd
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1")
dat_csv
```

La salida nos retorna:

Out[8]:

	id	Nombre	Sexo	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000
5	6	Pedro	m	10500
6	7	Gaby	f	11000
7	8	Cecilia	f	27000

Si queremos que en lugar de todos los datos, se restrinja la salida a las primeras cinco filas, podemos utilizar el método `head()`.

```
dat_csv.head()
```



Out[9]:

	id	Nombre	Sexo	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000

En los dos casos anteriores, aparecen los elementos de la primera fila como los nombres de las columnas de datos, si no quisiéramos este comportamiento podemos agregar el atributo `header = None`, de la siguiente manera:

```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1", header = None)
dat_csv
```

La salida nos retorna los datos de forma tabular y presentando solo las 5 primeras filas presentando la primera línea como nombres de columna ya que ya que hemos utilizado el método `head()`.

Out[1]:

	id	Nombre	Sexo	Sueldo
0	1	Juan	m	10000
1	2	Pedro	m	20000
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000

Si quisiéramos que tomar una determinada línea como cabecera, debemos asignarle al parámetro `header` el número de fila correspondiente, en el siguiente ejemplo se toma la línea 3 como cabecera:



```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1", header = 3)
dat_csv
```

Out[11]:

	3	Anna	f	10500
0	4	Julieta	f	13000
1	5	Pablo	m	30000
2	6	Pedro	m	10500
3	7	Gaby	f	11000
4	8	Cecilia	f	27000

Por defecto, **read_csv** asigna un índice numérico predeterminado que comienza con cero al leer los datos. Sin embargo, es posible alterar este comportamiento pasando el nombre de la columna que utilizaremos como índice. A continuación se dejara la columna **id** como índice de la tabla:

```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1", index_col='id')
dat_csv
```

Out[13]:

	Nombre	Sexo	Sueldo
id			
1	Juan	m	10000
2	Ramón	m	20000
3	Anna	f	10500
4	Julieta	f	13000
5	Pablo	m	30000

En el caso de que queramos restringir la tabla a algunas columnas específicas, podemos realizarlo con el parámetro **usecols** indicando en una lista las columnas seleccionadas.



```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1",  
usecols=['Nombre', 'Sueldo'])  
dat_csv.head()
```

Out[15]:

	Nombre	Sueldo
0	Juan	10000
1	Ramón	20000
2	Anna	10500
3	Julieta	13000
4	Pablo	30000

Nota: Si alguna de las líneas se encontraran en blanco, podemos eliminar dichas líneas mediante el atributo **skip_blank_lines=False**

Si queremos eliminar una fila en particular, podemos utilizar el parámetro **skiprows**, en el siguiente caso se eliminan las filas 1 y 4:

```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1", skiprows = [1,4])  
dat_csv
```

Out[16]:

	id	Nombre	Sexo	Sueldo
0	2	Ramón	m	20000
1	3	Anna	f	10500
2	5	Pablo	m	30000
3	6	Pedro	m	10500
4	7	Gaby	f	11000
5	8	Cecilia	f	27000

Para presentar un número dado de filas desde el inicio, podemos utilizar el parámetro **nrows**, en el siguiente ejemplo se presentan las primeras tres filas de datos.



```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1", nrows=3)
dat_csv
```

Out[19]:

	id	Nombre	Sexo	Sueldo
0	1	Juan	m	10000
1	2	Ramón	m	20000
2	3	Anna	f	10500

Para seleccionar un rango específico de filas podemos utilizar el método `query` especificando los límites aplicables sobre una determinada columna:

```
dat_csv = pd.read_csv('empleados.csv', encoding = "ISO-8859-1")
dat_csv.query('2 < id < 6')
```

Out[21]:

	id	Nombre	Sexo	Sueldo
2	3	Anna	f	10500
3	4	Julieta	f	13000
4	5	Pablo	m	30000

Para tratar con los datos de una columna, podemos recuperarlos dentro de una lista mediante el uso de un bucle `for`, y luego si es necesario convertir la lista en una array como se muestra a continuación:

```
import pandas as pd
import numpy as np
dat_csv = pd.read_csv('datos.csv', encoding = "ISO-8859-1")
datos_x = dat_csv.x
datos_y = dat_csv.y
x = []
y = []
for i in dat_csv.x:
    x.append(i)
for j in dat_csv.y:
```



```
y.append(j)
print(x)
print(y)

x_array = np.array(x)
print(type(x_array))
print(x_array)
```

La salida nos retorna:

```
[10000, 20000, 10500, 13000, 30000, 10500, 11000, 27000]
```

Desde Excel

En el caso de trabajar directamente con un archivo de Excel, podemos utilizar el método `read_excel()` de forma análoga a como lo hicimos anteriormente con `read_csv()`

```
import pandas as pd
dat_excel = pd.read_excel('empleados.xlsx')
dat_excel.head()
```

Nota: El parámetro **sheetname** nos permite seleccionar el número de hoja, las cuales se comienzan a numerar desde cero.

Formato json

El formato JSON es un lenguaje de intercambios de datos como XML pero mucho más simple. Es un lenguaje muy utilizado en el envío de datos desde aplicaciones web y móviles realizadas con javascript. Por su similitud con el tipo de datos de objetos de javascript a adquirido un papel muy importante en desarrolladores de esta comunidad. También es muy utilizado por los desarrolladores de videojuegos de y de animaciones para describir las partes del cuerpo de un avatar o darle formato a las texturas de un objeto.

El formato JSON para los que trabajamos con python sería muy similar asemejarlo a una lista de diccionarios, o sea que podemos pensar en la siguiente estructura:

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning



```
[  
  {  
    " ": " ",  
    " ": " ",  
    " ": " ",  
  },  
  {  
    " ": " ",  
    " ": " ",  
    " ": " ",  
  },  
]
```

De hecho cuando transformamos un json a python es entendido exactamente así, como una lista de diccionarios. Consideremos el siguiente ejemplo:

json1.json

```
1  {  
2    "nombre": "Ana",  
3    "edad": 33,  
4    "estado_civil": true,  
5    "esposo": "Pablo",  
6    "hijos": ["Cecilia", "Luis"],  
7    "autos": [  
8      {  
9        "modelo": "Ford",  
10       "color": "rojo"  
11      },  
12      {  
13        "modelo": "Chevrolet",  
14        "color": "azul"  
15      }  
16    ]  
17  }
```

Pandas utiliza el método `read_json()` para importar un archivo con este formato como se muestra a continuación:

```
import pandas as pd  
movies_json = pd.read_json('json1.json')
```

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning



`movies_json.head()`

Out[2]:

	nombre	edad	estado_civil	esposo	hijos	autos
0	Ana	33	True	Pablo	Cecilia	{'modelo': 'Ford', 'color': 'rojo'}
1	Ana	33	True	Pablo	Luis	{'modelo': 'Chevrolet', 'color': 'azul'}

3. Visualización con Matplotlib

Vamos a iniciar a trabajar con matplotlib aprendiendo a agregar comentarios y descripciones a los gráficos, para ello crearemos un gráfico con tres curvas en el dominio $x = [0, 2]$ y adicionaremos 100 puntos equiespaciados dentro del dominio mediante:

`x = np.linspace(0, 2, 100)`

Luego podemos crear las curvas utilizando el método `plot()` en donde pasamos como parámetros:

- Primero los puntos correspondientes a la coordenada en x.
- Segundo los puntos de la coordenada en y.
- Tercero el color de línea o el tipo de representación para los puntos de la línea, si en lugar de poner 'pink' ponemos '+' cada punto es representado por un signo de (+).
- Cuarto el atributo **label** con la descripción de la curva. Para que la descripción salga en pantalla debemos agregamos el método **legend()**.

```
plt.plot(x, x, 'pink', label='lineal')
plt.plot(x, x**2, label='cuadratica')
```

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning



```
plt.plot(x, x**3, label='cubica')
```

```
plt.legend()
```

Colores

Para los colores también tenemos otras opciones como:

- Dar el valor hexadecimal de forma análoga a como se hace en una página web dentro de código css. Los colores representan tres canales de colores:
 - Canal rojo #ff0000
 - Canal verde #00ff00
 - Canal azul #0000ff
- Se puede dar una abreviatura para el color, por ejemplo 'm' representa el color magenta.
- Se puede especificar el parámetro c con un valor de C0 al C9, lo cual agrega colores con buen contraste preestablecidos.
- Los colores pueden ser dados en canales de colores rgb en donde se dan tres parámetros entre 0 y 1 representando los canales rojo, verde y azul de la siguiente manera: $c=(1,0.1,1)$
- Los colores pueden ser dados en canales de colores rgba en donde se dan tres parámetros entre 0 y 1 representando los canales rojo, verde y azul y un cuarto parámetro que determina el nivel de opacidad, de la siguiente manera: $c=(1,0.1,1,0.5)$

Legenda

La legenda puede posicionarse en la pantalla si tomamos la longitud de cada eje como si estuviera en el intervalo **(0,1)**, por lo que podemos en base a esto agregar dentro de **legend()** la ubicación según cada eje con el uso del parámetro **loc**. De esta forma

podemos darle una posición específica dentro del gráfico de la siguiente forma:
plt.legend(loc=(0.5,0.7))

Descripciones de título y ejes

Para agregar descripciones a los ejes y título utilizamos los métodos:

- `xlabel()` para agregar una descripción en el eje x.
- `ylabel()` para agregar una descripción en el eje y.
- `title()` para adicionar un título al gráfico.

```
plt.xlabel('Eje X')  
plt.ylabel('Eje Y')  
plt.title("Estructura de gráfico")
```

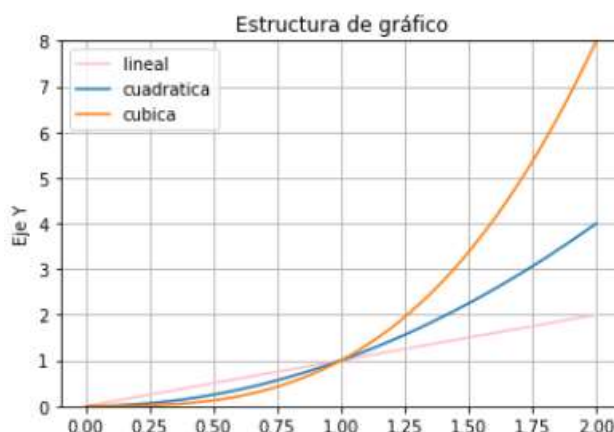
Limites en ejes

También podemos especificar los límites de la representación de datos en los ejes mediante:

- `plt.ylim(valor mínimo, valor máximo)` para el eje y.
- `plt.xlim(valor mínimo, valor máximo)` para el eje x.

```
plt.ylim(0,2)  
plt.ylim(0,8)
```

Si queremos adicionar una grilla lo podemos realizar mediante el método **grid()**. La representación visual nos queda:

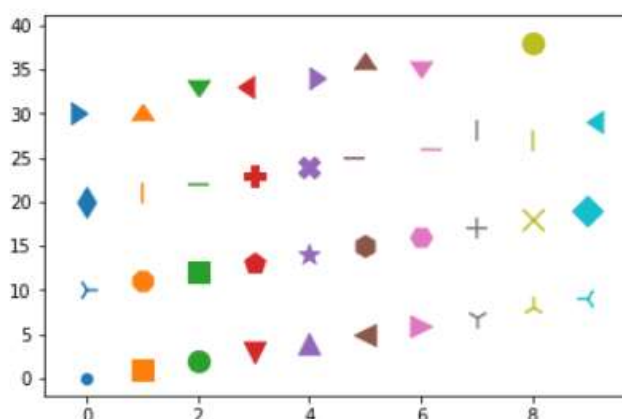


Tipos de marcas

En lugar de obtener puntos en las dispersiones, podemos agregar distintos tipo de indicadores, podemos verlos todos al ejecutar el siguiente código:

```
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

for i,marker in enumerate(Line2D.markers):
    plt.scatter(i%10,i,marker=marker,s=150)
plt.show()
```



El tamaño del indicador lo puedo modificar con el parámetro **s**.

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning



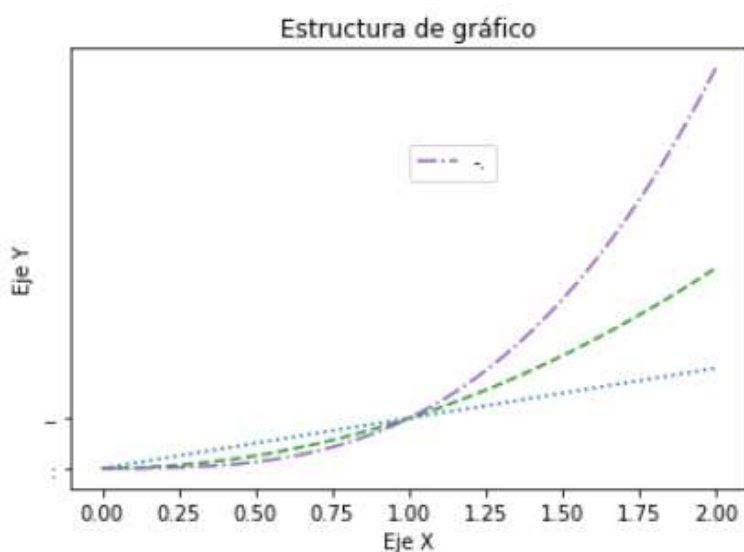
Estilos de línea

Podemos agregar diferentes tipos de trazado de línea utilizando los siguientes parámetros:

Parámetro	Tipo de línea
' - '	Línea llena
' -- '	Línea discontinua
' - . '	Línea de raya punto
' . '	Línea de puntos grandes
' : '	Línea de puntos

Veamos un ejemplo:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 2, 100)
plt.plot(x, x, 'r', '-')
plt.plot(x, x**2, 'b', '--')
plt.plot(x, x**3, 'g', '-.')
plt.xlabel('Eje X')
plt.ylabel('Eje Y')
plt.title("Estructura de gráfico")
plt.show()
```



Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning

4.- Mapa de colores

Cuando vamos a utilizar mapas de colores para representar nuestros datos, tenemos que tener en cuenta elegir aquel que mejor represente el conjunto de datos.

Los investigadores han encontrado que el cerebro humano percibe cambios en el parámetro de luminosidad como cambios en los datos mucho mejor que, por ejemplo, cambios en el tono. Por lo tanto, el espectador interpretará mejor los mapas de colores que tienen una luminosidad que aumenta monótonamente a través del mapa de colores.

Los mapas de colores a menudo se dividen en varias categorías según su función, por ejemplo:

- **Secuencial:** Es un cambio en la luminosidad y, a menudo, saturación del color de forma incremental, a menudo con un solo tono; Se debe utilizar para representar información que tiene ordenamiento.
- **Divergente:** Es un cambio en la luminosidad y posiblemente saturación de dos colores diferentes que se encuentran en el medio en un color insaturado; debe usarse cuando la información que se está trazando tiene un valor medio crítico, como la topografía o cuando los datos se desvían alrededor de cero.
- **Cualitativo:** A menudo son colores variados; debe utilizarse para representar información que no tiene orden o relaciones.

Secuencial

Veamos cómo trabajar con mapas del tipo secuencial, se puede encontrar una referencia completa de todos los tipos de mapas en el siguiente link:

<https://matplotlib.org/2.1.0/tutorials/colors/colormaps.html#mycarta-banding>

Para comprender el siguiente script debemos tener en mente que en matplotlib se denomina:

- **figure:** Al área que contiene la figura completa a representar.
- **axes:** a cada figura representada dentro de figure.

Centro de e-Learning SCEU UTN - BA.

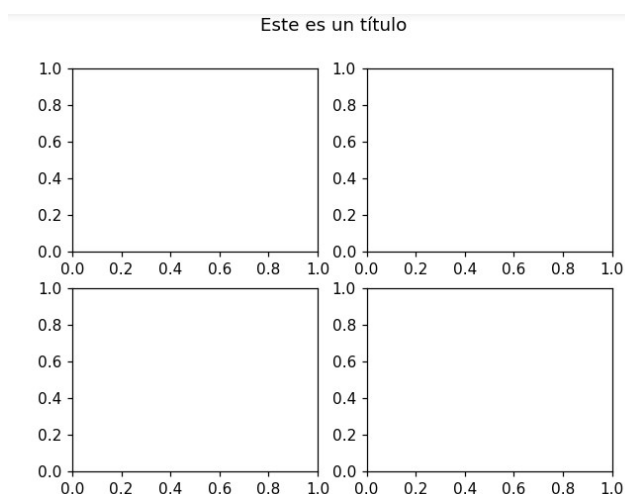
Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148
www.sceu.frba.utn.edu.ar/e-learning



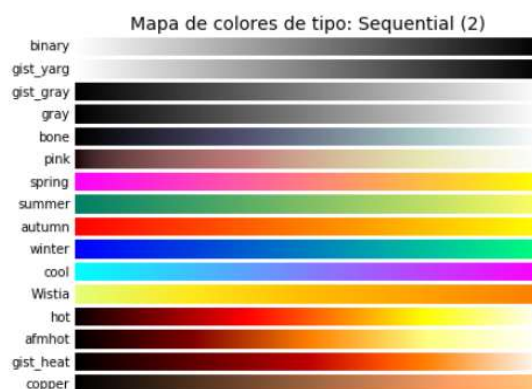
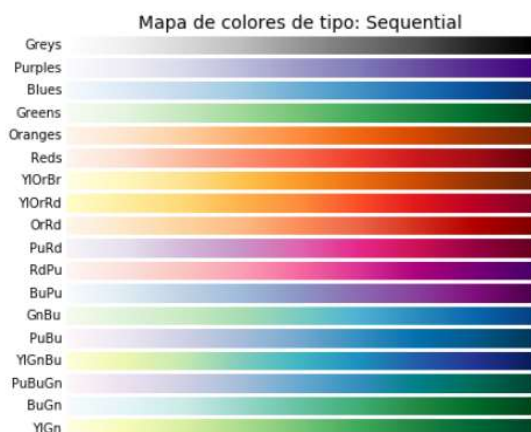
- axis: a los ejes de referencia de cada axes.
- figsize: es el tamaño de cada axes.

Asi por ejemplo si queremos crear cuadro figuras (axes) dentro de un área (gigu

```
import matplotlib.pyplot as plt
import numpy as np
fig, axes = plt.subplots(2, 2) # Figura con grillas de ejes 2x2
fig.suptitle('Este es un título')
```



En Matplotlib podemos obtener los siguientes mapas de colores:



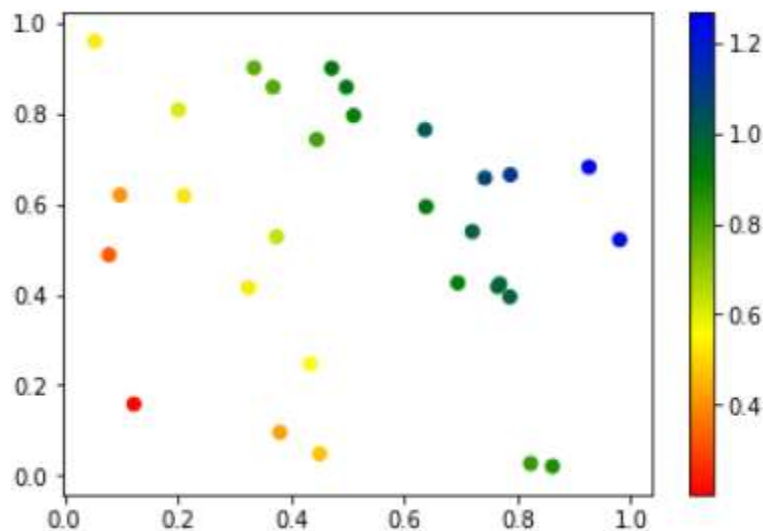


Un ejemplo muy útil de mapa de color, sería asignar colores en una escala según algún criterio específico, por ejemplo en el siguiente script se crea un mapa de valores aleatorios en donde se utiliza el valor en x mas el valor en y sobre dos para representar el color de la dispersión.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.colors
```

```
np.random.seed(7)
x,y = zip(*np.random.rand(50,2))
color=[]
print(len(x))
for i in range(0,50):
    color.append((x[i]+y[i])/2)
```

```
cmap = matplotlib.colors.LinearSegmentedColormap.from_list("", ["red","yellow","green",
"blue"])
plt.scatter(x,y,c=color, cmap=cmap)
plt.colorbar()
plt.show()
```





Bibliografía utilizada y sugerida

Libros

Programming Python 5th Edition – Mark Lutz – O'Reilly 2013

Programming Python 4th Edition – Mark Lutz – O'Reilly 2011

Manual online

<https://docs.python.org/3.7/tutorial/>



Lo que vimos

En esta unidad hemos visto como adquirir datos con pandas y como representar datos con Matplotlib.



Lo que viene:

En la siguiente unidad veremos algunas herramientas de sistema muy útiles, como formatear datos y fechas, y una introducción a la librería Turtle.